

CPSC-354 Report

Mitchell Toney
Chapman University

November 2, 2025

Abstract

Contents

1	Introduction	2
2	Week by Week	2
2.1	Week 1: HW1	2
2.2	Week 2: HW2	2
2.3	Week 3: HW3	4
2.3.1	Exercise 5	4
2.3.2	Semantic question	5
2.4	Week 4: HW4	6
2.4.1	HW4.1: Termination Proof for GCD	6
2.4.2	HW4.2: Termination Proof for Merge Sort	6
2.5	Week 5: HW5	7
2.5.1	Lambda Calculus Workout Evaluation (Corrected)	7
2.6	Week 6: HW6	7
2.6.1	Fixed Point Combinator Exercise	7
2.7	Week 7: HW 7 - Parse Trees Exercise	9
2.7.1	String: $2 + 1$	9
2.7.2	String: $1 + 2 * 3$	9
2.7.3	String: $1 + (2 * 3)$	10
2.7.4	String: $(1 + 2) * 3$	11
2.7.5	String: $1 + 2 * 3 + 4 * 5 + 6$	12
2.8	Week 8: HW 8	12
2.8.1	Exercise 1	12
2.9	Week 9: HW 9	13
2.9.1	No Induction	13
2.9.2	Induction	13
2.10	Week 10: HW 10	14
2.10.1	Level 6: Curry	14
2.10.2	Level 7: Un-Curry	14
2.10.3	Level 8: Go buy chips and dip!	14
2.10.4	Level 9: Uncertain Snacks	15
3	Essay	15
4	Evidence of Participation	15
5	Conclusion	15

1 Introduction

2 Week by Week

2.1 Week 1: HW1

The *MU* puzzle is a puzzle created by Douglas Hofstadter. It consists of four rules that can be applied to a string *MI*.

1. $xI \rightarrow xIU$
2. $Mx \rightarrow Mxx$
3. $xIIIy \rightarrow xUy$
4. $xUUy \rightarrow xy$

When first approaching this puzzle, the first strategy that came to mind was to take advantage of rule number 2 to keep duplicating the I's until there is a multiple of three, then using rules 3 and 4 to get rid of the I's and leave a remaining U.

The issue with this is that $2^n \bmod 3$ will never equal 0, it infinitely cycles between equaling 1 and 2, and without being able to get rid of all the I's, which would require them being a multiple of 3, you will never be able to get MU.

Thus, the puzzle is not solvable.

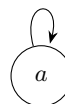
2.2 Week 2: HW2



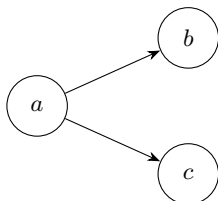
1. $A = \emptyset, R = \emptyset$



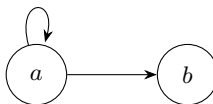
2. $A = \{a\}, R = \emptyset$



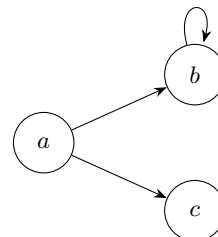
3. $A = \{a\}, R = \{(a, a)\}$



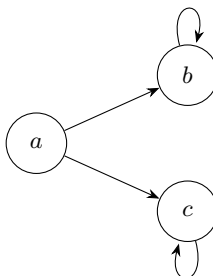
4. $A = \{a, b, c\}, R = \{(a, b), (a, c)\}$



5. $A = \{a, b\}, R = \{(a, a), (a, b)\}$



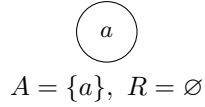
6. $A = \{a, b, c\}, R = \{(a, b), (b, b), (a, c)\}$



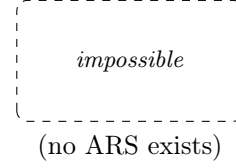
7. $A = \{a, b, c\}, R = \{(a, b), (b, b), (a, c), (c, c)\}$

#	Terminating	Confluent	Unique NFs
1	Yes	Yes	Yes
2	Yes	Yes	Yes
3	No	Yes	No
4	Yes	No	No
5	No	Yes	Yes
6	No	No	No
7	No	No	No

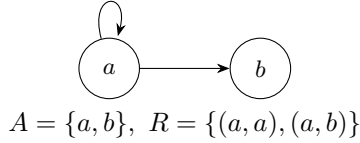
**Confluent True, Terminating True,
Unique NFs True**



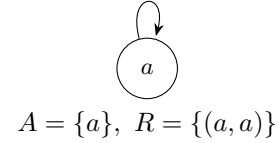
**Confluent True, Terminating True,
Unique NFs False**



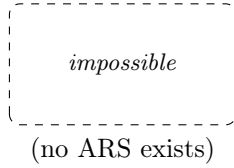
**Confluent True, Terminating False,
Unique NFs True**



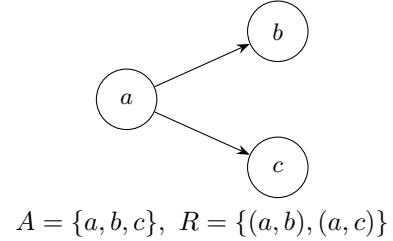
**Confluent True, Terminating False,
Unique NFs False**



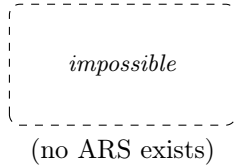
**Confluent False, Terminating True,
Unique NFs True**



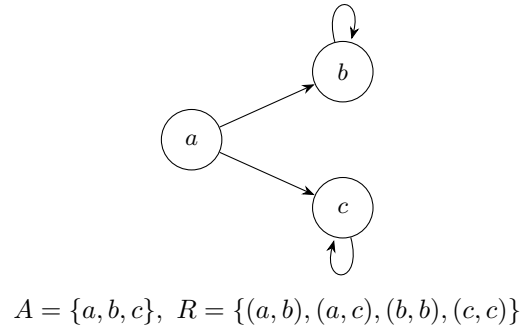
**Confluent False, Terminating True,
Unique NFs False**



**Confluent False, Terminating False,
Unique NFs True**



**Confluent False, Terminating False,
Unique NFs False**



2.3 Week 3: HW3

2.3.1 Exercise 5

Consider rewrite rules:

$$\begin{aligned}ab &\rightarrow ba \\ba &\rightarrow ab \\aa &\rightarrow \\b &\rightarrow\end{aligned}$$

Example Reductions

Reducing abba:

$$\begin{aligned}abba &\rightarrow baba \quad (\text{using } ab \rightarrow ba) \\baba &\rightarrow bbaa \quad (\text{using } ba \rightarrow ab) \\bbaa &\rightarrow baa \quad (\text{using } b \rightarrow \varepsilon) \\baa &\rightarrow aba \quad (\text{using } ba \rightarrow ab) \\aba &\rightarrow baa \quad (\text{using } ab \rightarrow ba)\end{aligned}$$

There is an infinite loop between **aba** and **baa**.

Reducing bababa:

$$\begin{aligned}bababa &\rightarrow ababab \quad (\text{using } ba \rightarrow ab) \\ababab &\rightarrow baabab \quad (\text{using } ab \rightarrow ba) \\baabab &\rightarrow ababab \quad (\text{using } ba \rightarrow ab)\end{aligned}$$

This is an infinite loop between **ababab** and **baabab**.

Why the ARS is not terminating The ARS is not terminating because the rules $ab \rightarrow ba$ and $ba \rightarrow ab$ create infinite cycles. These rules allow us to swap adjacent a and b characters indefinitely, leading to non-terminating reduction sequences.

Non-equivalent strings Two strings that are not equivalent: **a** and **aa**.

The string **a** cannot be reduced further, while **aa** reduces to nothing using the rule $aa \rightarrow$. Since *nothing* $\neq a$, these strings are in different equivalence classes.

Equivalence classes The equivalence relation $\leftrightarrow^*$ has infinitely many equivalence classes. Each equivalence class can be characterized by the number of a 's modulo 2 and the number of b 's modulo 1.

The equivalence classes are:

- $[\varepsilon]$: strings with even number of a 's and no b 's
- $[a]$: strings with odd number of a 's and no b 's
- $[b]$: strings with any number of a 's and at least one b

The normal forms are: ε , a , and b .

Modifying the ARS to be terminating To make the ARS terminating without changing equivalence classes, we can remove the symmetric rules and keep only one direction:

$$\begin{aligned}ba &\rightarrow ab \\aa &\rightarrow \varepsilon \\b &\rightarrow \varepsilon\end{aligned}$$

This eliminates the infinite cycles while preserving the same equivalence relation.

2.3.2 Semantic question

Parity of a 's: "Does this string contain an odd number of a 's?"

Answer: Yes if the normal form is a , No if the normal form is ε .

Exercise 5b

Consider rewrite rules:

$$\begin{aligned}ab &\rightarrow ba \\ba &\rightarrow ab \\aa &\rightarrow a \\b &\rightarrow\end{aligned}$$

Reducing abba:

$$\begin{aligned}abba &\rightarrow baba \\baba &\rightarrow abba\end{aligned}$$

Reducing bababa:

$$\begin{aligned}bababa &\rightarrow ababab \\ababab &\rightarrow bababa\end{aligned}$$

Why not terminating. The symmetric swaps $ab \leftrightarrow ba$ allow infinite rewriting.

Non-equivalent strings. a and ε are not equivalent.

Equivalence classes. Exactly two: $[\varepsilon]$ (no a 's; all b 's delete) and $[a]$ (at least one a ; since $aa \sim a$). Normal forms (under a terminating orientation): ε and a .

Terminating variant.

$$\begin{aligned}ba &\rightarrow ab \\aa &\rightarrow a \\b &\rightarrow\end{aligned}$$

This gives a complete semantics to the ARS: it computes the invariant for any input string.

2.4 Week 4: HW4

2.4.1 HW4.1: Termination Proof for GCD

Consider the following algorithm:

```
while b != 0:
    temp = b
    b = a mod b
    a = temp
return a
```

Assume: Work over integers with Euclidean division: inputs $a \in \mathbb{Z}$, $b \in \mathbb{N}$; if $b \neq 0$ then $0 \leq a \bmod b < b$.

Model: States $A = \mathbb{Z} \times \mathbb{N}$. One step $(a, b) \rightarrow (a', b')$ is one loop iteration.

Measure: $\phi : A \rightarrow \mathbb{N}$, $\phi(a, b) = b$.

Show: $(a, b) \rightarrow (a', b') \Rightarrow \phi(a', b') < \phi(a, b)$.

If $b \neq 0$, the update sets $b' = a \bmod b$ with $0 \leq b' < b$; hence $\phi(a', b') = b' < b = \phi(a, b)$.

ϕ strictly decreases in $\mathbb{N}$, so there is no infinite $\rightarrow$ -chain; eventually $b = 0$ and the loop stops. Thus the algorithm terminates under the stated conditions.

2.4.2 HW4.2: Termination Proof for Merge Sort

Consider the following fragment of an implementation of merge sort:

```
function merge_sort(arr, left, right):
    if left >= right:
        return
    mid = (left + right) / 2
    merge_sort(arr, left, mid)
    merge_sort(arr, mid+1, right)
    merge(arr, left, mid, right)
```

Prove that

$$\phi(\text{left}, \text{right}) = \text{right} - \text{left} + 1$$

is a measure function for `merge_sort`.

Show: For

```
merge_sort(arr, left, right):
    if left >= right: return
    mid = floor((left + right)/2)
    merge_sort(arr, left, mid)
    merge_sort(arr, mid+1, right)
    merge(arr, left, mid, right)
```

the function $\phi(\text{left}, \text{right}) = \text{right} - \text{left} + 1$ is a strictly decreasing measure, hence `merge_sort` terminates.

Assume: $\text{left}, \text{right} \in \mathbb{Z}$ with $0 \leq \text{left} \leq \text{right} < |\text{arr}|$. Division for `mid` is integer. If $\text{left} < \text{right}$, then $\text{left} \leq \text{mid} < \text{right}$. `merge` makes no recursive calls.

Model: States are intervals $A = \{(l, r) \in \mathbb{Z}^2 \mid l \leq r\}$. A "step" is a recursive edge from (l, r) (with $l < r$) to each child (l, mid) and $(\text{mid} + 1, r)$.

Measure: $\phi : A \rightarrow \mathbb{N}$, $\phi(l, r) = r - l + 1$.

Let $l < r$ and $\text{mid} = \lfloor (l + r)/2 \rfloor$ so $l \leq \text{mid} < r$.

First child: $\phi(l, \text{mid}) = \text{mid} - l + 1 \leq (r - 1) - l + 1 = r - l = \phi(l, r) - 1 < \phi(l, r)$.

Second child: $\phi(\text{mid} + 1, r) = r - \text{mid} \leq r - l = \phi(l, r) - 1 < \phi(l, r)$.

Every recursive edge strictly decreases ϕ in $\mathbb{N}$, which is well-founded. Thus no infinite recursion is possible; the calls bottom out at states with $\phi \in \{0, 1\}$ (i.e., `left` $\geq$ `right`), where the function returns. Therefore $\phi(\text{left}, \text{right}) = \text{right} - \text{left} + 1$ is a valid measure and `merge_sort` terminates under the stated conditions.

2.5 Week 5: HW5

2.5.1 Lambda Calculus Workout Evaluation (Corrected)

Evaluate: $(\lambda f. \lambda x. f(f\ x))(\lambda g. \lambda y. g(g\ y))$

Use α -renaming to avoid capture.

$$\begin{aligned} (\lambda f. \lambda x. f(f\ x))(\lambda g. \lambda y. g(g\ y)) &\rightarrow \lambda x. (\lambda g. \lambda y. g(g\ y))((\lambda g. \lambda y. g(g\ y))\ x) \\ &\quad (\lambda g. \lambda y. g(g\ y))\ x \rightarrow \lambda y. x(x\ y) \quad (\alpha\text{-rename inner } y) \\ \Rightarrow \lambda x. (\lambda g. \lambda y. g(g\ y))(\lambda y. x(x\ y)) &\rightarrow \lambda x. \lambda y. x^9 y. \end{aligned}$$

Hence the normal form is $\lambda f. \lambda x. f^9 x$ (Church numeral 9).

2.6 Week 6: HW6

2.6.1 Fixed Point Combinator Exercise

Compute `fact 3` following the computation rules for `fix`, `let`, and `let rec`:

Given:

$$E_0 = \text{let rec fact} = \lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } n * \text{fact } (n - 1) \text{ in fact } 3$$

Abbreviations:

$$F \stackrel{\text{def}}{=} \lambda f. \lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } n * f\ (n - 1) \qquad \text{FACT} \stackrel{\text{def}}{=} \text{fix } F$$

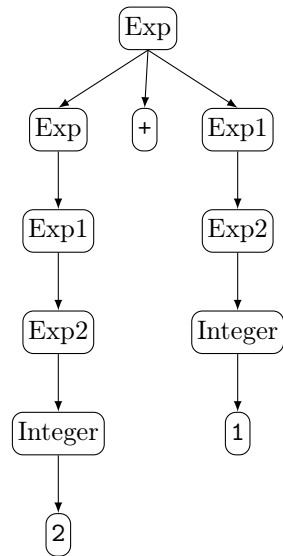
Computation:

$$\begin{aligned}
E_0 &\xrightarrow{\text{def of let rec}} \text{let fact} = \text{fix}(\lambda f. \lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)) \text{ in fact } 3 \\
&\xrightarrow{\text{def of let}} (\lambda \text{fact}. \text{fact } 3) \text{ fix}(\lambda f. \lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)) \\
&\xrightarrow{\beta} (\text{fix}(\lambda f. \lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1))) 3 \\
&\equiv \text{FACT } 3 \\
&\xrightarrow{\text{def of fix}} (\text{F FACT}) 3 \\
&\xrightarrow{\beta} (\lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } n * \text{FACT}(n-1)) 3 \\
&\xrightarrow{\beta} \text{if } 3 = 0 \text{ then } 1 \text{ else } 3 * \text{FACT}(3-1) \\
&\xrightarrow{\text{def of if}} 3 * \text{FACT}(3-1) \\
&\xrightarrow{\text{arith}} 3 * \text{FACT } 2 \\
&\xrightarrow{\text{def of fix}} 3 * (\text{F FACT}) 2 \\
&\xrightarrow{\beta} 3 * (\lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } n * \text{FACT}(n-1)) 2 \\
&\xrightarrow{\beta} 3 * \text{if } 2 = 0 \text{ then } 1 \text{ else } 2 * \text{FACT}(2-1) \\
&\xrightarrow{\text{def of if}} 3 * (2 * \text{FACT}(2-1)) \\
&\xrightarrow{\text{arith}} 3 * (2 * \text{FACT } 1) \\
&\xrightarrow{\text{def of fix}} 3 * (2 * (\text{F FACT}) 1) \\
&\xrightarrow{\beta} 3 * (2 * (\lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } n * \text{FACT}(n-1)) 1) \\
&\xrightarrow{\beta} 3 * (2 * \text{if } 1 = 0 \text{ then } 1 \text{ else } 1 * \text{FACT}(1-1)) \\
&\xrightarrow{\text{def of if}} 3 * (2 * (1 * \text{FACT}(1-1))) \\
&\xrightarrow{\text{arith}} 3 * (2 * (1 * \text{FACT } 0)) \\
&\xrightarrow{\text{def of fix}} 3 * (2 * (1 * (\text{F FACT}) 0)) \\
&\xrightarrow{\beta} 3 * (2 * (1 * (\lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } n * \text{FACT}(n-1)) 0)) \\
&\xrightarrow{\beta} 3 * (2 * (1 * \text{if } 0 = 0 \text{ then } 1 \text{ else } 0 * \text{FACT}(0-1))) \\
&\xrightarrow{\text{def of if}} 3 * (2 * (1 * 1)) \\
&\xrightarrow{\text{arith}} 3 * (2 * 1) \xrightarrow{\text{arith}} 3 * 2 \xrightarrow{\text{arith}} 6.
\end{aligned}$$

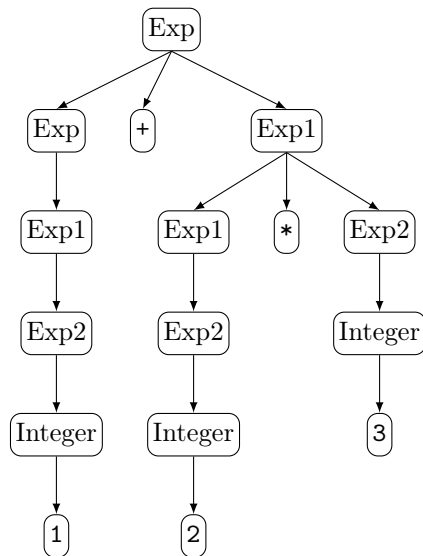
Result: fact 3 = 6

2.7 Week 7: HW 7 - Parse Trees Exercise

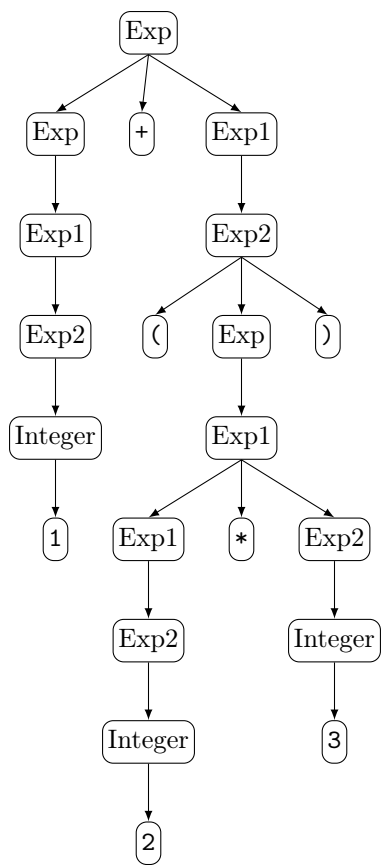
2.7.1 String: $2 + 1$



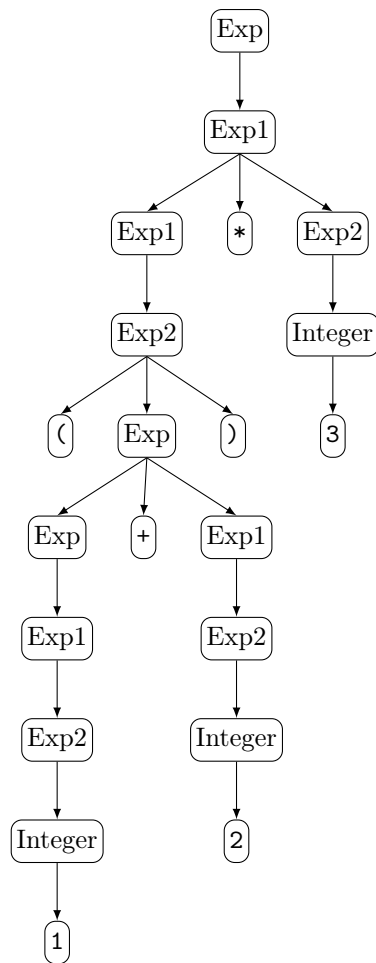
2.7.2 String: $1 + 2 * 3$



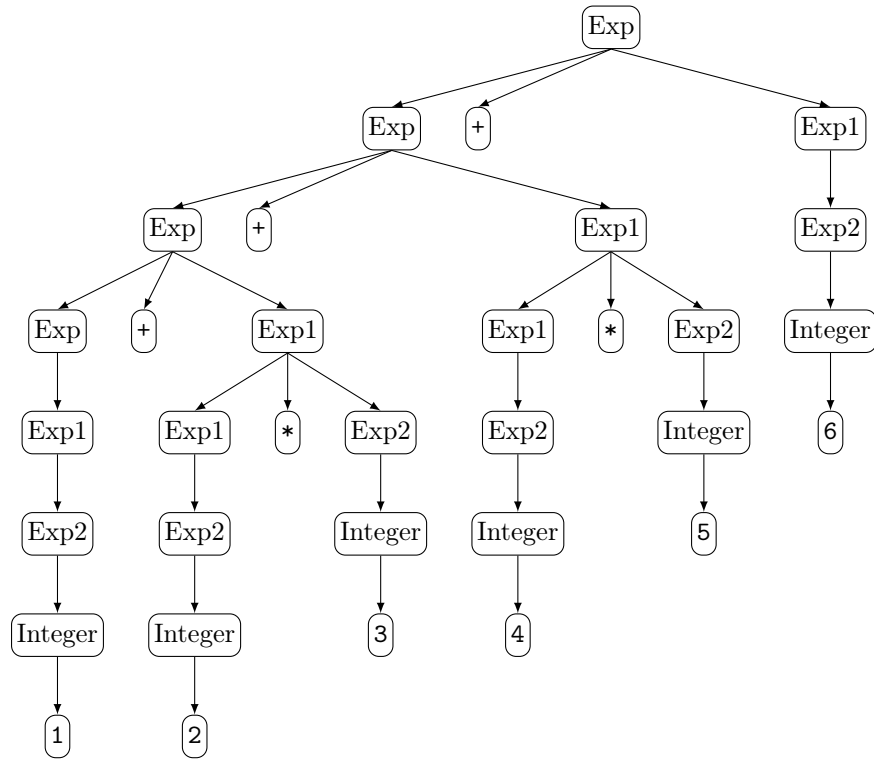
2.7.3 String: $1 + (2 * 3)$



2.7.4 String: $(1 + 2) * 3$



2.7.5 String: $1 + 2 * 3 + 4 * 5 + 6$



2.8 Week 8: HW 8

2.8.1 Exercise 1

Level 5:

$$a + (b + 0) + (c + 0) = a + b + c$$

```
rw [add_zero]
```

```
rw [add_zero]
```

rfl

Level 6:

$$a + (b + 0) + (c + 0) = a + b + c$$

```
rw [add_zero c]
```

```
rw [add_zero b]
```

rfl

Level 7:

$$succn = n + 1$$

```
rw [one_eq_succ_zero]
```

```
rw [add_succ]
```

```
rw[add_zero]
```

rfl

Level 8:

$$2 + 2 = 4$$

```

rw [two_eq_succ_one]
rw [four_eq_succ_three]
rw [three_eq_succ_two]
rw [two_eq_succ_one]
rw [one_eq_succ_zero]
rw [add_succ]
rw [add_succ]
rw [add_zero]
rfl

```

Level 7 Natural Language Proof:

$$\text{succ } n = n + 1$$

Replace 1 by succ 0:

$$\text{succ } n = n + \text{succ } 0$$

Use the definition of addition with a successor:

$$\text{succ } n = \text{succ}(n + 0)$$

Simplify $n + 0$ to n :

$$\text{succ } n = \text{succ } n$$

Both sides are identical, so the equality holds.

2.9 Week 9: HW 9

2.9.1 No Induction

```

rw [add_assoc a b c]
rw [add_comm b c]
rw [← add_assoc a c b]

```

For all $(a, b, c \in \mathbb{N})$,

$$\begin{aligned}
 a + b + c &= a + c + b. \\
 a + b + c &= a + (b + c) \quad (\text{associativity of } (+)) \\
 &= a + (c + b) \quad (\text{commutativity of } (+)) \\
 &= a + c + b \quad (\text{associativity of } (+)).
 \end{aligned}$$

2.9.2 Induction

```

induction c with d hd
rw [add_zero, add_zero]
rfl

```

```

rw [add_succ]
rw [add_succ]
rw [succ_add]
rw [hd]

```

For all $(a, b, c \in \mathbb{N})$,

$$a + b + c = a + c + b.$$

By induction on c .

Base case ($c = 0$).

$$\begin{aligned} a + b + 0 &= a + b && \text{(by } (x + 0 = x)) \\ &= a + 0 + b && \text{(by } (x + 0 = x)), \end{aligned}$$

Assume $a + b + d = a + d + b$:

$$\begin{aligned} a + b + (d + 1) &= (a + b + d) + 1 && \text{(by } (x + (y + 1) = (x + y) + 1)) \\ &= (a + d + b) + 1 && \text{(induction hypothesis)} \\ &= (a + d + 1) + b && \text{(by } ((x + y) + 1 = (x + 1) + y)) \\ &= a + (d + 1) + b && \text{(associativity).} \end{aligned}$$

Thus $a + b + c = a + c + b$ for all $c \in \mathbb{N}$.

2.10 Week 10: HW 10

2.10.1 Level 6: Curry

Objects: $C, D, S : \text{Prop}$

Assumptions: $h : C \wedge D \rightarrow S$

Goal:

$C \rightarrow D \rightarrow S$

Solution:

```
exact λ c d => h (and_intro c d)
```

2.10.2 Level 7: Un-Curry

Objects: $C, D, S : \text{Prop}$

Assumptions: $h : C \rightarrow D \rightarrow S$

Goal:

$C \wedge D \rightarrow S$

Solution:

```
exact λ cd => h (and_left cd) (and_right cd)
```

2.10.3 Level 8: Go buy chips and dip!

Objects: $C, D, S : \text{Prop}$

Assumptions: $h : (S \rightarrow C) \wedge (S \rightarrow D)$

Goal:

$S \rightarrow C \wedge D$

Solution:

```
exact λ s => and_intro (and_left h s) (and_right h s)
```

2.10.4 Level 9: Uncertain Snacks

Objects: $R, S : \text{Prop}$

Goal:

$$R \rightarrow (S \rightarrow R) \wedge (\neg S \rightarrow R)$$

Solution:

```
exact λ r => and_intro (λ s => r) (λ _ => r)
```

Question: Does currying versus uncurrying ever change definitional equality enough to affect the rewrites from the homework?

3 Essay

4 Evidence of Participation

5 Conclusion

References

[BLA] Author, [Title](#), Publisher, Year.